Charon: Fine-Grain Resource Allocation and Optimized Task Placement for Multi-Query Distributed Stream Processing

Donatien Schmitz
donatien.schmitz@uclouvain.be
UCLouvain
Louvain-la-Neuve, Belgium

Guillaume Rosinosky
guillaume.rosinosky@inria.fr
IMT Atlantique, Nantes Université,
École Centrale Nantes, CNRS, Inria,
LS2N - UMR 6004
F-44000 Nantes, France

Etienne Rivière
etienne.riviere@uclouvain.be
UCLouvain
Louvain-la-Neuve, Belgium

Abstract

Distributed Stream Processing (DSP) enables the analysis of large volumes of continuous data. A DSP engine generally executes multiple queries on a shared cluster. However, state-of-the-art DSP engines such as Apache Flink provision resources to queries in isolation and as one-size-fits-all units of CPU and memory, despite the variety of their requirements.

We propose Charon, a novel approach that dynamically assigns resources to processing elements at fine-grained levels and considers the complete set of colocated queries in its placement strategies, thereby improving consolidation and resource efficiency. Charon uses optimized placement strategies based on a bin-packing optimization algorithm, while respecting the throughput requirements of each query. We implement Charon in Flink and the Flink Kubernetes Operator. We evaluate our approach using queries from the Nexmark multi-query benchmark and demonstrate its effectiveness in improving resource utilization compared to Flink.

CCS Concepts

• **Computer systems organization → Data flow architectures**; • **General and reference → Performance**; **Estimation**; • **Information systems → Data streams**.

Keywords

Distributed Stream Processing, Scheduling, Resource Allocation, Multiple Query, Consolidation, Apache Flink

ACM Reference Format:

Donatien Schmitz, Guillaume Rosinosky, and Etienne Rivière. 2026. Charon: Fine-Grain Resource Allocation and Optimized Task Placement for Multi-Query Distributed Stream Processing. In *The 41st ACM/SIGAPP Symposium on Applied Computing (SAC '26), March 23–27, 2026, Thessaloniki, Greece.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3748522.3779716

1 Introduction

Distributed Stream Processing (DSP) engines, e.g., Apache Storm [5] or Apache Flink [4], are designed to process large volumes of data in real-time, making them essential for applications such as data analytics, machine learning, and the inspection of Internet of Things data [13]. A DSP engine is deployed on a large cluster and often supports multiple queries simultaneously, e.g., to support a data lake [23]. These queries typically differ in their complexity, the volume of processed data, and the nature of their computation, leading to widely varying resource requirements [14, 17, 32]. For instance, our recent analysis of a real-world industrial DSP workload [30] reveals that queries running in a representative data lake deployment based on Flink and Kafka are highly heterogeneous in terms of data volume, complexity, and resource requirements.

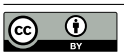
Motivation. DSP queries are composed of multiple *operators*, each of which can be executed in parallel across different nodes. This allows DSP systems to scale horizontally and distribute the event processing workload across multiple machines. To sustain a given throughput, elastic scaling policies analyze the resource usage of query operators and automatically add or retire instances [11, 28].

The allocation of resources to operator instances in current DSP systems uses a *one-size-fits-all* approach. Each instance receives a fixed amount of CPU and dedicated memory, and these amounts are uniform for all queries. This clashes with the diversity of needs experienced by operators, which depends on the nature of their computation and the workload they receive. For instance, some operators may be stateless, leading to wasted allocations of dedicated memory. Others may experience low traffic and use only a fraction of allocated CPU capacity when others saturate and must scale out.

Contributions. We make two complementary contributions towards more efficient resource usage in a multi-query DSP cluster.

Our first contribution is to enable autonomous fine-grained resource allocation for operator instances. Flink recently introduced the ability to define heterogeneous *slots* [6], i.e., computational and dedicated memory units for parallel operator instances, as previously proposed by R-Storm [25] for Apache Storm. While allowing the allocation of varying shares of CPU and memory, this mechanism suffers from two limitations. On one hand, resource allocation is static and only defined at compile time. On the other hand, there is no autonomous *policy* that allows determining the appropriate allocation for arbitrary operators, relying instead on prior expert knowledge. We address these limitations by (1) enabling the adaptation of resource allocation upon the deployment or re-configuration of a query, allowing an elastic scaling policy to decide not only on operator parallelism but also on the allocation of resources to its instances; and (2) designing a novel elastic scaling policy that extends the state-of-the-art DS2 [19] algorithm to decide on fine-grained allocations that respect the processing rate requirements of queries.

We further observe that typical placement strategies used in DSP, such as considering queries in isolation and deploying their operator instances in round-robin over all available nodes, result



SAC '26, Thessaloniki, Greece

ACM ISBN 979-8-4007-2294-3/2026/03
https://doi.org/10.1145/3748522.3779716

in suboptimal resource utilization when using fine-grained allocation. In contrast with one-size-fits-all allocation, where the number of computation *slots* at each node is fixed and all slots are equal, fine-grained allocation results in multiple, heterogeneous units of CPU and memory. A greedy placement is often suboptimal in such scenarios due to fragmentation. Our second contribution is thus a novel placement approach that (1) leverages a bin-packing algorithm to produce optimized placements of operator instances to nodes and maximize consolidation, i.e., the volume that can fit on a given set of nodes; and (2) considers multiple queries simultaneously when deciding on placement, rather than placing one query at a time. We evaluate two policies for placement: one that leverages common re-scaling triggers for multiple queries within the same observation period, without re-scaling stable queries, and one that may re-optimize the placement of all queries to maximize consolidation. These policies express trade-offs between the cost of reconfigurations and the resource usage, corresponding to different deployment scenarios (e.g., sets of short- vs long-lived queries).

We implement these contributions in a multi-query resource allocation and task placement solution that we call Charon, and that we integrate with Flink and the Flink Kubernetes Operator [7]. We evaluate Charon with the multi-query reference benchmark Nexmark [33] on an eight-node cluster. Our results show that our approach effectively reduces the number of containers required to support all queries, i.e., requiring only 43% to 60% of the number of containers otherwise necessary with the unmodified Flink.

Outline. The rest of the paper is structured as follows. Section 2 provides background information on DSP, Flink, and resource allocation and placement. Section 3 refines the identification of the shortcomings of existing approaches and outlines the principles of Charon. Sections 4 to 6 present the core components of Charon, namely its monitoring and trigger mechanisms, its resource allocation algorithms, and optimized task placement. Our large-scale evaluation Charon is presented in Section 7. We review related work in Section 8, and conclude in Section 9.

2 Background

We provide the necessary background on DSP, including resource management and elastic scaling. We focus on Apache Flink and its terminology, but other DSPs feature similar mechanisms [13].

Stream Processing Queries. A query is a logical, directed acyclic graph (DAG) of operators. Each operator inputs *events* on which it performs a specific processing task, such as filtering or aggregation. Some operators consume multiple streams, i.e., to perform unions and joins. Edges in the DAG are data streams between operators. Source and sink vertices respectively ingest and output events from and to external sources, such as Apache Kafka [27] or Kinesis [1].

Queries Execution. Figure 1 gives an example of two queries and their execution. Each query logical graph is transformed into a physical graph. Logical operators are generally merged when operating on the same data stream [15]. Each resulting physical operator is mapped to several parallel instances, and the input data stream is split across these instances (e.g., using hash partitioning).

In Flink, operator instances are named *tasks*. A Flink cluster comprises a Job Manager (JM) that coordinates the execution of tasks over several Task Manager (TM) nodes [10]. A TM supports several Task Slots (TS), each capable of executing a single, mono-threaded task. Each TS is assigned CPU and memory resources as defined in the TM configuration. A classical deployment strategy in Flink is to allocate one TS per available CPU core on the TM. Figure 1 illustrates the mapping from the two logical operator graphs to their respective physical plans and their subsequent deployment on three TMs. In this example, each TM is provisioned with 4 TS, each receiving 1 CPU core and a similar memory configuration.

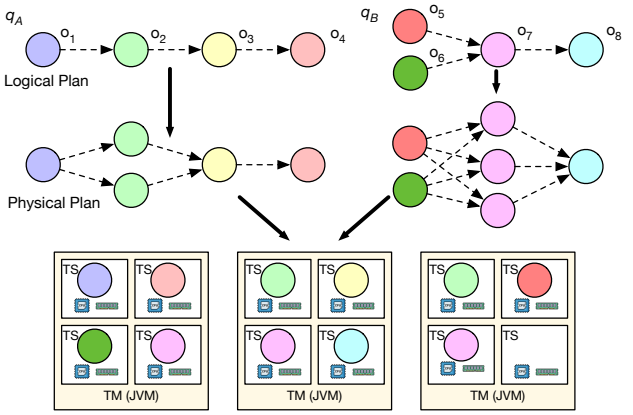


Figure 1: Deployment of two queries (q_A and q_B) on a Flink cluster. Queries logical graphs (top) are transformed into physical plans with multiple parallel tasks (middle). The tasks are deployed on the three Task Managers (TM), each with a fixed number of Task Slots (TS) (bottom). Each TS provides a one-size-fits-all share of CPU and memory.

Memory Management. TM memory is split into two main regions with different isolation guarantees across TS. Heap memory is managed by the supporting Java Virtual Machine (JVM) and is shared between TS. It is used for network memory (i.e., buffers) and temporary Java objects created during task execution. In contrast, managed memory is specific to each TS. In Flink recommended production settings, managed memory is used to persist the state of stateful operators (e.g., an aggregation or join) in combination with SSD storage using RocksDB [8]. This LSM-tree storage backend optimizes for write performance and reduces wear on the SSD. RocksDB utilizes managed memory to store write buffers, which are flushed to disk when they reach a target size (e.g., 64 MB), and a read cache for frequently accessed data.

Elastic Scaling. Elastic scaling (or *auto-scaling*) dynamically adjusts the parallelism of operators to accommodate an incoming rate of events. A vast literature exists on elastic scaling for DSP [11, 28] Flink uses a variant of the DS2 algorithm [19]. DS2 estimates the *busyness* of operators, i.e., the fraction of time their tasks spend processing events versus waiting for upstream operators. It also considers back-pressure, i.e., when a saturated operator slows down event processing rates at upstream operators. Periodically, the DS2 algorithm examines busyness and back-pressure metrics and may decide to re-scale, i.e., apply a new parallelism to some or all of a

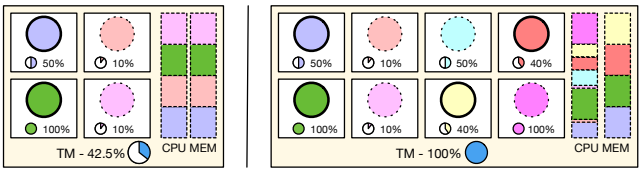


Figure 2: A one-size-fits-all approach to resource allocation can lead to suboptimal resource utilization. On the left, the default Flink configuration allocates one task per available CPU core to each task, leading to CPU underutilization and allocation of memory to stateless tasks (dotted circles). On the right, CPU is distributed as fine-grained shares, and memory is allocated only to tasks that require it (bold circles).

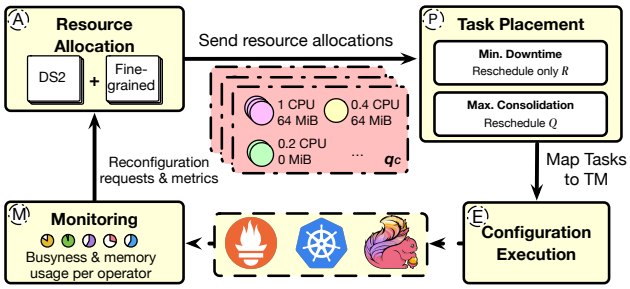


Figure 3: Charon components and workflow. Monitoring collects application metrics and triggers reconfigurations. Resource Allocation determines the optimal CPU and memory for query operators. Task Placement component then determines the optimal placement of tasks on the available TM. Finally, Charon applies the new resource allocation and task placement decisions to the running queries.

query's operators. The scaling decision is based on simple linearity assumptions for operators' capacities, and may require a few adjustment steps before converging to a stable configuration.

Elastic scaling in Flink is implemented in its Flink Kubernetes Operator [7] with metrics collected in Prometheus [3]. During re-scaling, operators are stopped, and a snapshot of the current state of stateful operators is taken from RocksDB [10] and stored in stable storage (e.g., HDFS). The JM then rebuilds the execution graph based on the scaling configuration, orders state redistribution between tasks, and restarts the query from the snapshot.

Task Placement. Task placement refers to the allocation of *tasks* across available TS and TM after elastic scaling. TS are homogeneous across all TM. Flink offers different task placement strategies, such as round-robin or selecting the TM with the lowest CPU occupancy. When the current set of TMs is full, the Flink Kubernetes Operator spawns a new TM as new Kubernetes *pods*.

3 Charon: Motivation and Overview

An essential limitation of Flink's *one-size-fits-all* resource allocation is that it provides the same amount of CPU and memory resources to each task, regardless of its actual requirements. Single-task operators with a busyness level lower than 100% are assigned a full CPU core, thereby wasting computational capacity. The allocation of managed memory to stateless operators is also a pure waste, as their tasks make no use of the RocksDB storage backend. Figure 2 (left) illustrates how these limitations result in an overall underused TM that is nonetheless unable to host new tasks.

Recently, Flink introduced a new feature allowing fine-grain resource allocation to tasks, i.e., selecting arbitrary amounts of managed and heap memory, and assigning *shares* of CPU resources lower than a full core [6]. With this feature, the number of TS on a TM is no longer fixed, but is limited only by the total assigned volume of each resource. Figure 2 (right) shows the potential of fine-grain resource allocation, where only stateful tasks are assigned managed memory and all are assigned CPU shared corresponding to their busyness levels. The resulting TM hosts more tasks than with the default resource allocation strategy, leading to higher resource utilization. While instrumental in allowing fine-grained resource management, this new mechanism suffers from two limitations. First, the allocation is fixed and declared at compile time, disallowing runtime adaptation. Second, the choice of resource amounts relies on expert prior knowledge. The appropriate resources for an operator depend, however, not only on the nature of its computation, but also on the volume and content of events it processes, which may only be known at runtime.

We also observe that existing simple and greedy task *placement* mechanisms are ill-suited for fine-grained resource allocation. First, they either do not consider the heterogeneity of tasks (e.g., for the Round Robin policy) or only consider a single dimension (e.g., the least-used TM policy only considers CPU allocation). Second, they target the placement of tasks *in isolation*, i.e., one after the other, rather than optimizing the placement of the multiple queries found in a typical DSP cluster. As a result, these placement policies achieve poor *consolidation*, and may require more TM to support a given workload than an optimized placement.

Overview. We present Charon, an integrated scheduler for multi-query DSP workloads that addresses these limitations. Charon builds upon two main principles: (1) autonomous fine-grained resource allocation integrated with state-of-the-art elastic scaling mechanisms; and (2) optimized, multi-query task placement mechanisms that aim to maximize consolidation.

We implement Charon using version 1.18 of Flink and 1.11 of the Flink Kubernetes Operator, but its principles apply to DSP systems that follow a similar computational model. Figure 3 presents an overview of its components, which are then detailed in the following three sections of this paper.

Charon extends Flink Monitoring with the ability to detect improper memory configurations (Ⓜ, Section 4). These metrics may trigger periodic re-scaling operations for some of the running queries. The Resource Allocation module extends elastic scaling policies with the ability to assign different CPU shares and memory allocations to different operators (Ⓐ, Section 5). The Task Placement module considers the resulting set of tasks and their requirements, and determines a placement that maximizes consolidation, thereby limiting the number of necessary TMs. In contrast with existing Flink greedy task placement strategies, this module considers the placement as a multi-dimensional optimization problem solved using a bin packing algorithm. We present two possible

Table 1: Notations used by Charon for an allocation and placement workflow initiated at the end of observation period t.

Symbol	Description
Default configurations	
C	Number of cores per TM
H	Available heap memory per TM
M	Default amount of Managed Memory per TS
Workload	
$q \in Q$	A query in the set of queries
$o \in O$	An operator in the set of all operators
$O_q \subseteq O$	Operators specific to query q
Inputs of Resource Allocation	
$\mathcal{R}^t \subseteq Q$	Set of queries requiring reconfiguration
b_o^t	Average observed busyness for operator o during last period
m_o^t	Use of managed memory (binary) by operator o during last period
Output of Resource allocation and inputs of Task Placement	
$\mathcal{A}^t$	Resource allocation, including:
$\mathcal{A}^t.\pi_o$	Parallelism (number of tasks) for operator o
$\mathcal{A}^t.c_o$	Share of CPU core(s) for each task of operator o (%)
$\mathcal{A}^t.m_o$	Share of managed memory for each task of operator o (%)
Output of Task Placement	
N^t	Number of necessary TM
$\mathcal{P}^t$	Map between tasks of operators O and TM $\{1, \ldots, N^t\}$

optimization policies that differ in their ability to redeploy stable queries to optimize consolidation, albeit at the cost of additional reconfiguration downtime (Ⓟ, Section 6). Finally, Charon applies the new resource allocation and task placement decisions to the running queries (Ⓔ). For the latter, we modified Flink and added new fine-grain allocation APIs allowing us to adjust resource allocation at runtime, upon a query reconfiguration.

Table 1 summarizes the notations used in the rest of the paper.

4 Monitoring

Monitoring collects information about running queries. The two metrics of interest are: (1) the busyness of operators, i.e., how much time is spent processing events *vs.* waiting for input by their tasks, as already used by the DS2 [19] algorithm and, in addition, (2) measures of the use of managed memory by these tasks, indicating access to the RocksDB state backend, or its absence thereof.

Charon targets minimal modification to Flink and the Flink Kubernetes Operator. This applies to the periodic metrics collection and aggregation strategy. While metrics are collected by the Prometheus service at a high frequency (every five seconds by default), they are aggregated over a longer period (one minute by default) to avoid noise and short-term fluctuations. We define $t > 1$ as the index of the last aggregation period. For all operators $o \in O$ currently deployed in the cluster, $b_o^t \in [0, 100\%]$ is the average busyness of all tasks of o, expressed as a fraction of time, for period t. Additionally, we observe the use of managed memory by examining the statistics of RocksDB access times. If these statistics are void while events were processed, we deduce that the task is stateless. This information is encoded in the binary variable m_o^t.

Monitoring is also responsible for triggering reconfiguration for queries that are considered under- or over-provisioned. We use the existing Flink mechanism, which is based on thresholds on b_o over the last observation period. A low average busyness indicates over-provisioning and the need to scale down the query containing operator o. Symmetrically, a high average busyness indicates under-provisioning and the need to scale out the corresponding query. At the end of period t, operators crossing thresholds $b_o^t \leq 20\%$ or $b_o^t \geq 80\%$ flag the corresponding queries for reconfiguration in a set $\mathcal{R}^t \subseteq Q$, i.e., $\mathcal{R}^t = \{q \in Q \mid \exists o \in O_q, b_o^t \leq 20\% \vee b_o^t \geq 80\%\}$. The choice of these thresholds is based on experimental analysis that yielded good reconfiguration responsiveness without excessive oscillations.

Reconfiguration triggers are decided at the end of every one-minute observation period. Following a reconfiguration request and the restart of queries by the resource allocation and task placement modules, which we detail next, Monitoring skips two observations as a cool-down period to allow the system to stabilize.

Algorithm 1: Charon resource allocation policy

Parameters: The previous configuration $\mathcal{A}^{t-1}$
Parameters: Query set $\mathcal{R}^t$ requiring reconfiguration
Result: A new configuration $\mathcal{A}^t$

```
1  A^t ← A^{t-1}  // Keep configuration for non reconfigured queries.
2  for q ∈ R^t do
3      for o ∈ O_q do
4          A^t.π_o ← DS2(o)   // Fetch the recommended parallelism.
5          if ¬m_o^t                          // Operator is stateless?
6              A^t.m_o ← 0%                   // Disable managed memory.
7          else
8              A^t.m_o ← 100% × M             // Default managed memory.
9          end if
10         if A^t.π_o = 1 ∧ A^{t-1}.π_o = 1    // Stays single-task?
11             A^t.c_o ← ⌈b_o^t / 20%⌉ × 20%  // Allocate fraction of CPU.
12         else
13             A^t.c_o ← 100%                 // Keep full core.
14         end if
15     end for
16 end for
17 return A^t        // Return new configuration for task placement.
```

5 Resource Allocation

Resource Allocation combines elastic scaling with the determination of optimal resource allocation for operators of queries selected for reconfiguration, i.e., in set $\mathcal{R}^t$. In addition to selecting the number of instances using existing Flink algorithms, it can adapt the allocation of CPU and memory for different operators. The goal of Resource Allocation is to ensure that each operator has sufficient resources to handle its workload while minimizing resource waste and maximizing overall system efficiency.

Resource Allocation builds upon the existing elastic scaling algorithm in Flink, based on DS2 [19]. Based on Monitoring metrics, Resource Allocation adjusts the results of the elastic scaling algorithm to remove managed memory for stateless operators and adapts CPU allocation for low-traffic operators.

Algorithm 1 details Resource Allocation operation when triggered at the end of period t. Inputs are the set of queries under reconfiguration $\mathcal{R}^t$, the average busyness of their operators b_o^t, and their use of managed memory m_o^t. Its output is a mapping $\mathcal{A}^t$ between each operator in these queries and to its resource allocation, i.e., its parallelism ($\mathcal{A}^t.\pi_o$), CPU cores ($\mathcal{A}^t.c_o$), and amount of managed memory ($\mathcal{A}^t.m_o$).

The algorithm starts from a copy of the previous configuration (Line 1), then loops over the set of queries in $\mathcal{R}^t$, and every operator of these queries. For each operator, it fetches DS2's recommended parallelism (Line 4). If the operator is stateless, it disables its managed memory by setting it to 0% of the default allocation (Line 6); stateful operators keep 100% of that default value (Line 8). We note that *source* operators' parallelism is constant, as it depends on the query initial configuration and number of connections to data sources (e.g., Kafka or Kinesis). Source operators are generally stateless and can get stripped of managed memory with no performance penalty.

In a second phase, the algorithm checks whether the CPU allocation is currently oversubscribed and adjusts it to a fraction of a core if possible. We only consider operators with a parallelism of one, i.e., a single task, over at least two observation periods. For this, we verify that the parallelism in the observation period $t-1$ was already one (Line 10). In this case, the algorithm allocates a fraction of a CPU core to the operator o based on its observed busyness level b_o^t (Line 11). This allocation is in discrete fractions to avoid very small CPU shares but also to make the process of task placement, detailed in the next section, less computationally expensive. Operators that do not match these criteria keep the default allocation of one CPU core (Line 13). The allocation of heap memory to tasks is the same as their allocation of CPU: a task gets $\mathcal{A}^t.c_o \times \frac{H}{C}$ heap memory, where H is the volume of heap memory for a TM and C its number of available cores.

Resource allocation outputs the new configuration $\mathcal{A}^t$ for all operators in the set of queries $\mathcal{Q}$ (Line 17), which will be the input to the task placement component we detail next.

6 Optimized Task Placement

After Resource Allocation determines the CPU and memory profiles for operators and the number of tasks they require, Task Placement is responsible for spawning these tasks on available TM, potentially launching new ones if necessary.

Flink's task placement strategies are designed with homogeneous tasks in mind (i.e., tasks with identical resource requirements), and deploy queries in isolation. As mentioned in Section 3, these strategies are ill-suited for heterogeneous requirements and result in sub-optimal consolidation.

Charon optimizes task placement, improves consolidation, and reduces the number of necessary TM by two means: (1) it does not use greedy placement policies but models task placement as a bin-packing optimization problem, and uses a solver to derive an optimal solution; and (2) it considers the reconfiguration of multiple queries *together* rather than one by one, thereby offering greater potential for improved consolidation.

We consider two policies, differing in the set of queries considered for placement. The *Minimum Downtime (MD)* policy only considers queries scheduled for reconfiguration ($\mathcal{R}$), but does not modify the placement of other queries ($\mathcal{Q} \setminus \mathcal{R}$). The rationale for this policy is to impose no downtime beyond what is already imposed by Monitoring decisions. It is suitable for setups where queries are relatively short-lived and/or where the cost of query downtime is deemed higher than the cost of using slightly more resources. In contrast, the *Maximum Consolidation (MC)* policy favors an optimal usage of these resources but does so by potentially replacing all running queries ($\mathcal{Q}$), and not only those scheduled for reconfiguration. It is adapted for workloads formed of long-running queries, where the cost of short-term reconfigurations and associated downtimes is compensated by lower resource utilization in the long term.

Both strategies take as input $\mathcal{R}$, $\mathcal{Q}$, and the resource allocation $\mathcal{A}^t$. They output the number of necessary TM, N^t, and a mapping of tasks to these TM, $\mathcal{P}^t$. Both strategies are modeled as bin packing problems. A bin packing problem is an optimization problem in which items (tasks) are placed into bins (TM) of a given capacity (CPU and memory). The objective is to place every item in the bins in a way that minimizes the number of necessary bins. Let n be the total number of tasks to place, and m be an upper-bound on the number of required TM, i.e., when each task receives a full core among the C available on each TM:

$$n = \sum_{q \in \mathcal{Q}} \sum_{o \in O_q} \mathcal{A}^t.\pi_o \tag{1}$$

$$m = \left\lceil \frac{n}{C} \right\rceil \tag{2}$$

Let $x_i \in \{0,1\}$ be a decision variable equal to 1 if TM i is used, and 0 otherwise. Moreover, let $y_{ij} \in \{0,1\}$ be a decision variable equal to 1 if task j is assigned to TM i and 0 otherwise, with $i \in \{1,\dots,m\}$ and $j \in \{1,\dots,n\}$. Finally, let P_{ij} be the set of pre-placed tasks to TM, i.e., tasks of queries not scheduled for reconfiguration in $\mathcal{R}^t$, where P_{ij} contains task j if it is currently assigned to TM i. To simplify notations, we note $q(j)$ the query to which a task j belongs and $o(j)$ its operator (thus, $\mathcal{A}^t.c_{o(j)}$ is the CPU allocation of j). The MD policy is equivalent to solving the following Integer Linear Program (ILP):

$$\text{Min.} \quad \sum_{i=1}^{m} x_i \tag{3}$$

$$\text{S. t.} \quad \sum_{j=1}^{n} \mathcal{A}^t.c_{o(j)} \cdot y_{ij} \le C \cdot x_i, \quad \forall i \in \{1,...,m\} \tag{4}$$

$$\sum_{j=1}^{n} \mathcal{A}^t.m_{o(j)} \cdot y_{ij} \le (M \cdot C) \cdot x_i, \quad \forall i \in \{1,...,m\} \tag{5}$$

$$\sum_{i=1}^{m} y_{ij} = 1, \quad \forall j \in \{1,...,n\} \tag{6}$$

$$y_{ij} = 1, \quad \forall i \in \{1,...,m\}, \forall j \in P_i \text{ if } q(j) \notin \mathcal{R} \tag{7}$$

$$x_i, y_{ij} \in \{0,1\} \tag{8}$$

The objective function (3) minimizes the number of necessary TM. The first two constraints, (4) and (5), ensure that the total CPU and memory requirements of tasks assigned to each TM do not exceed its capacity. The third constraint (6) ensures that each task is assigned to exactly one TM. Finally, the fourth constraint (7) pre-assigns tasks of queries not in $\mathcal{R}$ to their current TM assignation, avoiding unnecessary reconfigurations.

The MC policy gets rid of the fourth constraint (7) to allow all queries ($\mathcal{Q}$) to be replaced if this leads to a better placement. A query that has at least one task of one operator replaced will incur a snapshot and re-deployment of the query, and an associated

Table 2: Default throughput of the three streams used by the six queries of Nexmark (in events per second).

Event source	Q1	Q2	Q3	Q5	Q8	Q11
Auction	—	—	600	—	600	—
Person	—	—	200	—	200	—
Bid	9,200	9,200	—	9,200	—	9,200

downtime. However, this opens the possibility of more aggressive optimization and resource savings.

7 Evaluation

We evaluate the Charon prototype with the goal of answering the following research questions (RQ):

- RQ1: What is the impact of Charon fine-grained resource allocation? Considering one query, does it allow reducing the necessary resources for that query while matching rate requirements, and does it require fewer reconfigurations?
- RQ2: Does the multi-query, optimized placement of Charon improve consolidation compared to existing single-query policies used by Flink?
- RQ3: What is the relative impact of using the MD and MC strategies on consolidation and, for MC, on the number of reconfigurations?
- RQ4: Is the optimization approach used for task placement scalable?

Our metrics of interest are the number of necessary TM and the number of reconfigurations required to support target query throughputs. We compare the following four candidates:

- Baseline (**BL**) is the default Flink and Flink Kubernetes Operator resource allocation and placement.
- Fine Grained (**FG**) uses fine-grained resource allocation strategies as proposed in Section 5, but keeping the default (round-robin) placement strategy.
- Minimum Downtime (**MD**) and Maximum Consolidation (**MC**) use fine-grained resource allocation and optimized, multi-query placement with the two strategies outlined in Section 6.

Benchmarks. We use the well-established Nexmark benchmark [33]. Nexmark uses a collection of queries to simulate an online auction system. These queries have varying characteristics, including complexity, input rate, and resource requirements (e.g., some are fully stateless while others have complex stateful operators). We use the same queries as for the evaluation of DS2 [19]: **Q1**, **Q2**, **Q3**, **Q5**, **Q8**, and **Q11**. **Q1** and **Q2** are stateless queries and are primarily composed of respectively a *map* and a *filter* operator. **Q3** and **Q8** are stateful queries, with **Q3** performing an *incremental join* query and **Q8** computing a *join* over a window. **Q5** and **Q11** are both stateful queries performing *windowed aggregations*.

Nexmark uses three event sources — *Auction*, *Bid*, and *Person*. The benchmark determines the rate of event generation for these sources as a ratio: for every *Person* event generated, 3 *Auction* events and 46 *Bid* events are generated. The default rate is 10,000 events per second, with a distribution across sources and use by the six queries shown in Table 2. We note this default rate as "x1" and consider higher data volumes by multiplying the rates in Table 2 by 5 ("x5") and 10 ("x10").

Experimental Setup. We run the experiments on a testbed of eight nodes, each with 18 CPU cores and 96 GiB of RAM. One node is used solely for managing the Kubernetes cluster and for the observability stack, which includes Prometheus [3] and Grafana [2]. We use another dedicated node for the JM, and the remaining 6 nodes for TM. Each TM is configured with 4 CPU cores — resulting in 4 TS for the default one-size-fits-all resource allocation and placement strategy — and 4 GiB of RAM. The number of TM that are actually deployed depends on the decisions of the task placement algorithm.

7.1 Fine-Grained Resource Allocation: Impact on Individual Queries

We first address RQ1 and the impact of fine-grained resource allocation on the performance and cost for individual queries. Table 3 presents the resource utilization of queries after the elastic scaling process has converged, for both the BL and FG setups (i.e., using the default round-robin placement strategy). Each query runs for a total of 20 minutes, targeting the three throughput levels (x1, x5, and x10).

For each query, we report the final, stable resource allocation sustaining the target throughput, with the number of necessary TM, the number of tasks (TS), allocated CPU cores (C), and managed memory used by the query (M in MiB). We first observe that, in all configurations, the number of tasks (i.e., the operators' parallelism) is identical for BL and FG. For stateless queries (**Q1** and **Q2**), FG uses significantly fewer resources, both in terms of CPU and memory, than BL. Observations from Monitoring allow Resource Allocation to detect and remove unnecessary managed memory from stateless operators. For stateful query **Q3**, FG manages to reduce the number of TM used from 2 to 1, while using the same number of TS. For **Q5**, FG uses one less TM than BL throughout levels x1 and x5 but requires the same number of TM for x10, although using fewer CPU cores and memory overall. **Q8** requires a single TM for both candidates at every throughput level, although here again, FG uses fewer CPU cores and memory than BL. Finally, for **Q11**, FG uses one less TM than BL at level x1 but requires the same number of TM for x5 and x10, primarily due to CPU requirements. This more complex stateful query nonetheless manages to use fewer resources overall in all configurations.

In summary, this first series of experiments allows answering positively to RQ1: fine-grain resource allocation enables more efficient resource utilization for each query.

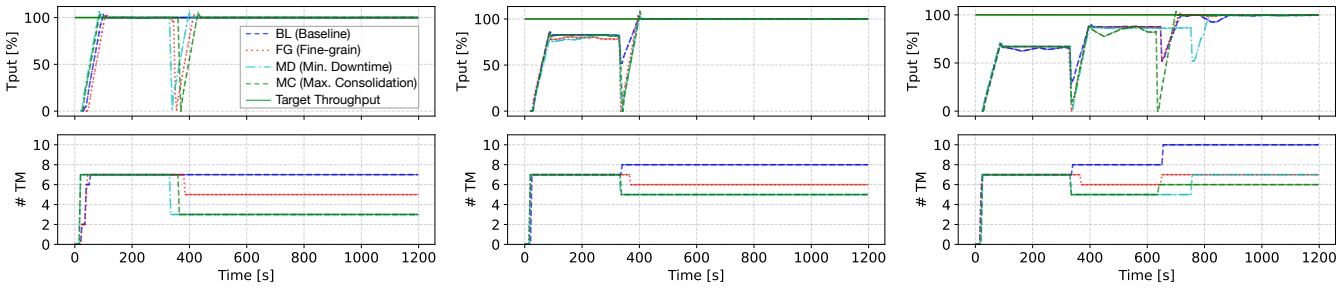
7.2 Placement with Multiple Queries

We now explore RQ2 and RQ3 with multi-query workloads, i.e., where the six Nexmark queries are running simultaneously and consume events from the same input flows. The total number of events consumed by the six queries combined is, respectively, 38,400 (x1), 192,000 (x5), and 384,000 (x10) events per second. We consider the four candidates: BL and FG, as in the previous subsection, plus the multi-query-aware MD and MC.

Figure 4 presents the achieved throughput (top) and the number of TM used (bottom) as a function of time, and for the three throughput levels. For readability, the throughput is normalized as

Table 3: Comparison of of the baseline (BL) and fine-grain (FG) scaling strategies for single queries. The queries run individually at different throughput levels. The values indicate for each *sustained* configuration: the number of needed TMs (TM), the total number of tasks (TS), the total number of CPU cores (C), and the total amount of managed memory used by the query (M in MiB). Bold values indicate improvements compared to the baseline.

Rate	Cand.	Q1				Q2				Q3				Q5				Q8				Q11			
		TM	TS	C	M	TM	TS	C	M	TM	TS	C	M	TM	TS	C	M	TM	TS	C	M	TM	TS	C	M
10k (x1)	BL	1	3	3	1029	1	3	3	1029	2	5	5	1715	2	5	5	1715	1	3	3	1029	2	5	5	1715
	FG	1	3	**1.4**	**0**	1	3	**1.4**	**0**	**1**	5	**2.6**	**343**	**1**	5	**3.4**	**686**	1	3	**2.2**	**343**	**1**	5	**3.4**	**686**
50k (x5)	BL	1	3	3	1029	1	3	3	1029	2	5	5	1715	3	9	9	3087	1	3	3	1029	4	15	15	5145
	FG	1	3	**1.4**	**0**	1	3	**1.4**	**0**	**1**	5	**2.6**	**343**	**2**	9	**7.4**	**2058**	1	3	**2.2**	**343**	4	15	**13.4**	**4116**
100k (x10)	BL	1	3	3	1029	1	3	3	1029	2	5	5	1715	4	15	15	5145	1	3	3	1029	7	27	27	9261
	FG	1	3	**1.4**	**0**	1	3	**1.4**	**0**	**1**	5	**2.6**	**343**	4	15	**13.4**	**4116**	1	3	**2.2**	**343**	7	27	**25.4**	**8232**



(a) Throughput: 38,400 events/s (x1) **(b) Throughput: 192,000 events/s (x5)** **(c) Throughput: 384,000 events/s (x10)**

Figure 4: Multi-query results for 4 scaling candidates for 3 throughput levels. The top-most plot of each figure shows the total throughput of all queries as a percentage of the target throughput (indicated by a dashed green line). The bottom plots show the number of Task Managers (TM) used by each candidate to achieve their throughput.

Table 4: Number of reconfigurations made by each candidate to reach the target throughput for each query and in total.

Rate	Cand.	Q1	Q2	Q3	Q5	Q8	Q11	Total
10k (x1)	BL	0	0	0	0	0	0	0
	FG	1	1	1	1	1	1	6
	MD	1	1	1	1	1	1	6
	MC	1	1	1	1	1	1	6
50k (x5)	BL	0	0	0	1	0	1	2
	FG	1	1	1	1	1	1	6
	MD	1	1	1	1	1	1	6
	MC	1	1	1	1	1	1	6
100k (x10)	BL	0	0	0	2	0	2	4
	FG	1	1	1	2	1	2	8
	MD	1	1	1	2	1	2	8
	MC	2	2	2	2	2	2	12

a percentage of the target throughput, which is represented by a dashed green line. Reconfigurations impact the throughput of some queries (with BL, FG, and MD) or all queries (with MC), followed by a ramp-up phase to reach the new, sustainable throughput. Table 4 presents the number of reconfigurations for the different candidates and the different throughputs.

The x1 scenario shows that the BL candidate does not reconfigure any of the queries and continues to use the original 7 TM used for one-size-fits-all provisioning at the beginning. In contrast, the three other candidates perform such reconfigurations and significantly reduce the number of TM while respecting the target throughput. In all cases, a single re-scaling decision is necessary. We can observe the impact of multi-query, optimized placement by comparing the results of FG on the one hand with those of MC and MD on the other hand. The latter two candidates manage to reduce the number of necessary TM to 3, i.e., a ≃43% of what the default Flink requires.

For the x5 throughput, BL scales only **Q5** and **Q11** once and ends up using 8 TM. FG, MD, and MC all reconfigure all queries once, and reduce the number of necessary TM through better consolidation. However, we observe the impact of placement optimization at time 350s, where MD and MC manage to place the resulting tasks on 5 TM while the round robin placement of FG needs 6 TM.

The x10 throughput is more demanding and requires more than one reconfiguration per query to sustain the target throughput. BL reconfigures all queries once, except **Q5** and **Q11** that need two reconfigurations, reaching a final resource budget of 10 TM. At time 350s, the three other candidates find the same configuration sustaining an overall throughput of ~85% of the target throughput. However, FG does not consolidate well and uses one more TM than MD and MC. After a final scaling step at 650s, FG achieves the target throughput using 7 TM, MD uses 7 TM, and MC uses 6 TM, 60% of the budget used by BL.

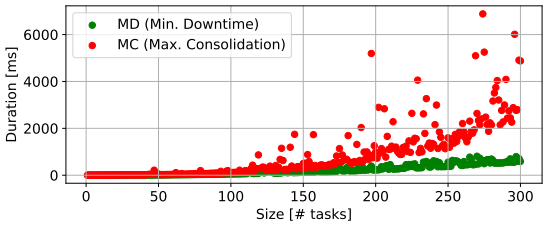


Figure 5: Solver efficiency for the scheduling problem with varying numbers of tasks. The solver's performance degrades as the number of tasks increases.

As expected, we observe that BL makes the least number of reconfigurations, as it only scales queries that cannot sustain the target throughput with their initial configuration. For throughput x1 and x5, FG, MD, and MC make the same number of reconfigurations for each query. Finally, in scenario x10, MC makes more reconfigurations than FG and MD, as it tries to optimize consolidation by redeploying tasks from already well-placed queries.

In summary, this experiment shows the interest of optimizing placement considering multiple queries compared to only using fine-grain resource allocation with the default, round-robin placement algorithm of Flink, positively answering RQ2. The MC strategy, by forcing the replacement of stable queries together with those identified for reconfiguration, achieves slightly better consolidation and resource utilization at the cost of additional reconfigurations than MD, also answering RQ3.

7.3 Task Placement Performance

We finally investigate RQ4 and the scalability of the task placement optimization problem. Both MD and MC candidates utilize bin-packing optimization, which is known to be NP-complete. Our implementation uses Google's OR-Tools [26] to solve the corresponding ILPs. We leverage the constraint programming capabilities of OR-Tools to efficiently explore the solution space and find optimal solutions for both strategies.

To evaluate the solver's efficiency, we conduct microbenchmarks on the placement problem with large numbers of heterogeneous tasks. We compare the performance of MC and that of MD with a pre-assignment of 25% of stable tasks to TM. Figure 5 shows the solver's performance as the number of tasks increases for MD and MC. The solver's performance degrades as the number of tasks increases, with a significant increase in solving time for instances with more than 200 tasks for MC, which is already a significant number for a DSP cluster. Pre-assignment helps reduce the search space and allows for higher task counts, as it provides additional constraints that guide the solver towards feasible solutions more quickly. A possibility to mask the cost for very large clusters and query sets would be to pre-compute placement and resource allocation before enacting decisions, i.e., outside of the auto-scaling decision loop, although this would require more significant changes to Flink and the Flink Kubernetes Operator. It is also possible to interrupt the search after a timeout duration for an approached solution. Overall, we conclude this evaluation by answering RQ4 affirmatively for common workload sizes.

8 Related Work

We review work on resource allocation and operator placement.

Elastic scaling and resource allocation. The problem of allocating sufficient parallelism to the operators of a stream processing query has been extensively studied in the literature. Surveys by Roger *et al.* [28] and by Cardellini *et al.* [11] provide comprehensive overviews of the topic. DS2 [19] is the reference algorithm integrated in Flink, and discussed in earlier sections. DS2, as most other elastic scaling algorithms, is based on linear capacity scaling assumptions for operators. For some workloads, an imbalance in the popularity of keys used to dispatch events to operators' tasks, generally referred to as *skew*, may jeopardize this assumption. Rebalancing mechanisms address this problem [13, 21].

Predictive scaling mechanisms aim to determine, prior to production deployment, the necessary scaling and resource profiles for DSP queries. OrientStream [34] or ZeroTune [9] leverage machine learning to predict the reaction of different operators to workload characteristics. StreamBed [29] uses Bayesian optimization to drive a series of small deployments of a query and model its supported throughput on a target amount of resources.

All the works discussed earlier consider a homogeneous allocation of resources to different operators. Justin [31] is a recent proposal integrating with Flink that extends DS2 [19] and the Flink Kubernetes Operator to decouple the allocation of CPU and memory to tasks and enable fine-grain resource allocation. In contrast with Charon, Justin only considers a single query deployment.

DSP execution and placement. Early work on task placement includes the SODA by the IBM Systems team [38], which proposes heuristic strategies for placing multiple queries onto a cluster while optimizing for load balancing and consolidation; or the work of Xing *et al.* [39] that propose algorithms for *resilient* placements, based on estimations of operators' capacities to handle short-term bursts and models of operators load scaling.

Q-Flink [16] and CAPSys [37] consider the *shared resource contention* problem, where co-located tasks of DSP queries compete for the same types of resources on a deployment node (e.g., CPU or I/O). Both works model the impacts of co-location and solve an optimization problem to derive a placement that minimizes contention. Similarly, Fan *et al.* [12] proposed a fine-grained sub-partitioning of DSP graphs to reduce resource contention and minimize the number of worker nodes. All these approaches consider a single query deployment, and are therefore complementary to Charon.

Finally, P-Deployer [22] targets the deployment of stream processing in cloud environments with the goal of minimizing inter-node communication. Similarly to Charon, the placement problem is solved using a bin-packing algorithm. P-Deployer does not consider, however, fine-grain resource allocation.

Multi-query optimization. Multi-query optimization has been extensively studied for relational databases, with advanced algorithms that determine, before deploying a set of queries, their common views, indexes, or plans. A recent survey by Zinchenko and Ponomaryov [40] gives a comprehensive overview of this field. Similar techniques were proposed for non-relational data, e.g., RDF data in SPARQL [20] or Map/Reduce jobs [35]. Finally, STREAMLINE [24]

is a system that predicts workload and continuously tunes the configuration of multiple running DSP queries, focusing on parallelism and buffer sizes.

Pre-deployment multi-query optimization for DSP is considered in State-Slice [36], where common or closely related computations across queries are merged into a single operator. Jonathan *et al.* [18] consider a similar approach for DSP queries deployed in a geo-distributed setting. These approaches are complementary to Charon: collections of merged or fused queries, even after optimization, can still benefit from multi-query runtime-optimized resource allocation and placement.

9 Conclusion

We presented Charon, a novel approach for DSP scheduling combining fine-grained resource allocation with optimized multi-query task placement. By dynamically adjusting CPU and memory allocations based on operator needs and modeling placement as a bin-packing problem, Charon achieves significant improvements in resource efficiency and consolidation over the state of the art. Experiments with the Nexmark benchmark show its ability to reduce wasted resources, lower the number of required TM, and balance trade-offs between downtime and consolidation.

Acknowledgments: This research was funded by the Walloon region (Belgium) through the Win2Wal project "GEPICIAD" and by a gift from Eura Nova. Experiments presented in this paper were carried out using the Grid'5000 testbed, supported by a scientific interest group hosted by Inria and including CNRS, RENATER and several Universities as well as other organizations (see https://www.grid5000.fr).

References

[1] Amazon Kinesis. https://aws.amazon.com/fr/kinesis/data-streams/, 2013.
[2] Grafana. https://grafana.com/docs/, 2014.
[3] Prometheus. https://prometheus.io, 2024.
[4] Apache Flink. https://flink.apache.org/, 2025.
[5] Apache Storm. https://storm.apache.org/, 2025.
[6] Flink Fine-Grained Resource Management. https://nightlies.apache.org/flink/flink-docs-master/docs/deployment/finegrained_resource/, 2025.
[7] Flink K8S Operator. https://github.com/apache/flink-kubernetes-operator, 2025.
[8] RocksDB. https://rocksdb.org/, 2025.
[9] Pratyush Agnihotri, Boris Koldehofe, Paul Stiegele, Roman Heinrich, Carsten Binnig, and Manisha Luthra. Zerotune: learned zero-shot cost models for parallelism tuning in stream processing. In *2024 IEEE 40th International Conference on Data Engineering*, ICDE, 2024.
[10] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. State management in Apache Flink®: consistent stateful distributed stream processing. *Proceedings of the VLDB Endowment*, 10(12):1718–1729, 2017.
[11] V. Cardellini, F. Lo Presti, M. Nardelli, and G. Russo. Runtime adaptation of data stream processing systems: The state of the art. *ACM Computing Surveys*, 54(11s):1–36, 2022.
[12] Yinuo Fan, Dawei Sun, Minghui Wu, Shang Gao, and Rajkumar Buyya. A fine-grained task scheduling strategy for resource auto-scaling over fluctuating data streams. *Future Generation Computer Systems*, 175(108119), February 2025.
[13] Marios Fragkoulis, Paris Carbone, Vasiliki Kalavri, and Asterios Katsifodimos. A survey on the evolution of stream processing systems. *The VLDB Journal*, 33(2):507–541, 2024.
[14] Luis Garcés-Erice, Sean Rooney, and Zoltán A Nagy. Analysis of SQL workloads on an enterprise datalake. In *2020 IEEE 13th International Conference on Cloud Computing*, CLOUD'20, 2020.
[15] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. A catalog of stream processing optimizations. *ACM Computing Surveys (CSUR)*, 46(4):1–34, 2014.
[16] M Reza HoseinyFarahabady, Ali Jannesari, Javid Taheri, Wei Bao, Albert Y Zomaya, and Zahir Tari. Q-Flink: A QoS-aware controller for Apache Flink. In *20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing*, CCGRID, 2020.
[17] Windsor W Hsu, Alan Jay Smith, and Honesty C. Young. Characteristics of production database workloads and the TPC benchmarks. *IBM Systems Journal*, 40(3):781–802, 2001.
[18] Albert Jonathan, Abhishek Chandra, and Jon Weissman. Multi-query optimization in wide-area streaming analytics. In *ACM symposium on cloud computing*, ACM SoCC, 2018.
[19] Vasiliki Kalavri, John Liagouris, Moritz Hoffmann, Desislava Dimitrova, Matthew Forshaw, and Timothy Roscoe. Three steps is all you need: fast, accurate, automatic scaling decisions for distributed streaming dataflows. In *13th USENIX Symposium on Operating Systems Design and Implementation*, OSDI, 2018.
[20] Wangchao Le, Anastasios Kementsietsidis, Songyun Duan, and Feifei Li. Scalable multi-query optimization for SPARQL. In *2012 IEEE 28th International Conference on Data Engineering*, 2012.
[21] Gang Liu, Zeting Wang, Amelie Chi Zhou, and Rui Mao. Adaptive key partitioning in distributed stream processing. *CCF Transactions on High Performance Computing*, 6(2):164–178, 2024.
[22] Xunyun Liu and Rajkumar Buyya. Performance-oriented deployment of streaming applications on cloud. *IEEE Transactions on Big Data*, 5(1):46–59, 2017.
[23] Fatemeh Nargesian, Erkang Zhu, Renée J Miller, Ken Q Pu, and Patricia C Arocena. Data lake management: challenges and opportunities. *Proceedings of the VLDB Endowment*, 12(12):1986–1989, 2019.
[24] Stefan Pedratscher, Zahra Najafabadi Samani, Juan Aznar Poveda, Thomas Fahringer, Marlon Etheredge, Abolfazl Younesi, Juan Jose Durillo Barrionuevo, and Peter Thoman. STREAMLINE: Dynamic and resource-efficient auto-tuning of stream processing data pipeline ensembles. *Internet of Things*, 2025.
[25] Boyang Peng, Mohammad Hosseini, Zhihao Hong, Reza Farivar, and Roy Campbell. R-storm: Resource-aware scheduling in Storm. In *ACM/IFIP Middleware*, 2015.
[26] Laurent Perron and Frédéric Didier. CP-SAT solver. https://developers.google.com/optimization/cp/cp_solver/.
[27] Theofanis P Raptis and Andrea Passarella. A survey on networked data streaming with Apache Kafka. *IEEE Access*, 11, 2023.
[28] H. Röger and R. Mayer. A comprehensive survey on parallelization and elasticity in stream processing. *ACM Computing Surveys*, 52(2):1–37, 2019.
[29] Guillaume Rosinosky, Donatien Schmitz, and Etienne Rivière. Streambed: capacity planning for stream processing. In *18th ACM International Conference on Distributed and Event-based Systems*, DEBS, 2024.
[30] Donatien Schmitz, Luc Berrewaerts, Guillaume Rosinosky, Sabri Skhiri, and Etienne Rivière. A tale of many streams: Characterizing a hybrid batch-stream production workload in Digazu, a data lake supported by Apache Kafka and Flink. In *19th ACM International Conference on Distributed and Event-based Systems*, DEBS, 2025.
[31] Donatien Schmitz, Guillaume Rosinosky, and Etienne Rivière. Justin: Hybrid CPU/Memory elastic scaling for distributed stream processing. In *IFIP International Conference on Distributed Applications and Interoperable Systems*, DAIS, 2025.
[32] Sacheendra Talluri, Alicja Łuszczak, Cristina L Abad, and Alexandru Iosup. Characterization of a big data storage workload in the cloud. In *ACM/SPEC International Conference on Performance Engineering*, ICPE, 2019.
[33] Pete Tucker, Kristin Tufte, Vassilis Papadimos, and David Maier. Nexmark – a benchmark for queries over data streams. Technical report, Oregon Health and Science University, 2008.
[34] Chunkai Wang, Xiaofeng Meng, Qi Guo, Zujian Weng, and Chen Yang. Orientstream: A framework for dynamic resource allocation in distributed data stream management systems. In *25th ACM International on Conference on Information and Knowledge Management*, ACM CIKM, 2016.
[35] Guoping Wang and Chee-Yong Chan. Multi-query optimization in MapReduce framework. *Proceedings of the VLDB Endowment*, 7(3):145–156, 2013.
[36] Song Wang, Elke Rundensteiner, Samrat Ganguly, and Sudeept Bhatnagar. State-slice: New paradigm of multi-query optimization of window-based stream queries. In *32nd international conference on Very large data bases*, VLDB, 2006.
[37] Yuanli Wang, Lei Huang, Zikun Wang, Vasiliki Kalavri, and Ibrahim Matta. CAPSys: Contention-aware task placement for data stream processing. In *20th European Conference on Computer Systems*, EuroSys, 2025.
[38] Joel Wolf, Nikhil Bansal, Kirsten Hildrum, Sujay Parekh, Deepak Rajan, Rohit Wagle, Kun-Lung Wu, and Lisa Fleischer. Soda: An optimizing scheduler for large-scale stream-based distributed computer systems. In *ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing*, Middleware, 2008.
[39] Ying Xing, Jeong-Hyon Hwang, Ugur Cetintemel, and Stan Zdonik. Providing resiliency to load variations in distributed stream processing. In *32nd international conference on Very large data bases*, VLDB, 2006.
[40] Sergey Zinchenko and Denis Ponomaryov. The selection problem in multi-query optimization: a comprehensive survey. *arXiv preprint arXiv:2412.11828*, 2024.